OCR API & Next.js Deployment on Ubuntu — Chat Transcript

28 October 2026

You

```
[2026-10-28 13:15:37,686] [ WARNING] install_check.py:60 - You are using GPU version PaddlePaddle, but there is no GPU
detected on your machine. Maybe CUDA devices is not set properly.
```

Claude

That's PaddlePaddle telling you the **GPU build** is installed, but it can't find a usable GPU at runtime. It's only a warning: Paddle falls back to the CPU, so your code still runs, just slower. What to do depends on whether the machine actually has an NVIDIA GPU.

If there's no GPU on this machine, switch to the CPU build so the warning goes away:

```
pip uninstall -y paddlepaddle-gpu
pip install paddlepaddle
```

If there is a GPU, work through these checks:

1. **Confirm the driver sees it.** Run `nvidia-smi`. If that fails, the NVIDIA driver isn't installed or loaded. In Docker, start the container with `--gpus all`; without that flag the container can't see the GPU.
2. **Check `CUDA_VISIBLE_DEVICES`.** If it's set to an empty string or `-1`, all GPUs are hidden. Run `echo $CUDA_VISIBLE_DEVICES`, then `unset CUDA_VISIBLE_DEVICES` or `export CUDA_VISIBLE_DEVICES=0`.
3. **Match versions.** The `paddlepaddle-gpu` wheel is built for a specific CUDA and cuDNN version. Compare the version `nvidia-smi` reports, and your installed CUDA toolkit, against the version your Paddle wheel expects. If they don't match, reinstall Paddle using the command for your CUDA version from the selector on paddlepaddle.org.cn.
4. **Library path.** If CUDA is installed in a non-standard location, make sure Paddle can find it. For example: `export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH`

Then verify:

```
import paddle
paddle.utils.run_check()
print(paddle.device.get_device())   # should print gpu:0
```

If you paste the output of `nvidia-smi` and `pip list | grep paddle`, I can tell you exactly which of these is the problem.

You

```
(mvenv) anand@anand:/opt/ocr_api$ nvidia-smi
Command 'nvidia-smi' not found, but can be installed with:
```

Claude

That means the NVIDIA driver isn't installed. The next step is to find out whether the machine has an NVIDIA GPU at all:

```
lspci | grep -i nvidia
```

If that prints nothing, there's no NVIDIA GPU, which is common on cloud VMs and most laptops or servers without a dedicated card. Switch to the CPU build inside your venv and you're done:

```
pip uninstall -y paddlepaddle-gpu
pip install paddlepaddle
```

If you're using PaddleOCR, it runs fine on CPU. If you pass `use_gpu=True` anywhere in your code, set it to `False` or remove it.

If it does list an NVIDIA card, install the driver (Ubuntu):

```
sudo ubuntu-drivers autoinstall
sudo reboot
```

After the reboot, `nvidia-smi` should work and show a "CUDA Version". Send me that output along with `pip list | grep paddle`, and I'll check that your Paddle build matches it.

You

i have nvidia gpu

Claude

Then you need to install the driver first. On Ubuntu:

```
# 1. Confirm the card is visible and see which driver is recommended
lspci | grep -i nvidia
ubuntu-drivers devices

# 2. Install the recommended driver
sudo ubuntu-drivers autoinstall      # or: sudo apt install nvidia-driver-XXX (the "recommended" one from above)

# 3. Reboot
sudo reboot
```

After the reboot, run `nvidia-smi`. You should see your GPU and a "CUDA Version" in the top-right corner. That number is the highest CUDA version your driver supports.

A few things can go wrong here:

- **Secure Boot.** If it's enabled, Ubuntu may ask you to set a password during install and then "Enroll MOK" on the blue screen at reboot. If you skip that step, the driver won't load and `nvidia-smi` will say it can't communicate with the driver.
- **Remote machine.** If it's a server you're connected to over SSH, make sure you can get back in after the reboot.
- **Docker.** If the OCR API runs in a container, you also need the NVIDIA Container Toolkit, and you must start the container with `--gpus all`.

Once `nvidia-smi` works, send me its output and the output of:

```
pip list | grep -i paddle
```

Your `paddlepaddle-gpu` build has to target a CUDA version that is no higher than the one `nvidia-smi` shows. If it's higher, I'll give you the exact reinstall command.

You

```
This site can't be reached
192.168.70.139 refused to connect.
```

Claude

"Refused to connect" means the machine at 192.168.70.139 is up and reachable, but nothing is listening on the port you're trying to reach. After a reboot, the usual reason is that your OCR API didn't start again on its own.

On the server, check whether anything is listening:

```
sudo ss -tlnp
```

If your API's port isn't in the list, the app isn't running. Start it again the way you normally do, for example:

```
cd /opt/ocr_api
source mvenv/bin/activate
python app.py        # or: uvicorn main:app --host 0.0.0.0 --port 8000
```

If it crashes on startup, paste the error here. It may be related to the new driver.

If the port is listed but shows `127.0.0.1:PORT`, the app only accepts local connections. Start it with `--host 0.0.0.0`, or set `host="0.0.0.0"` in `app.run()`.

If it's listening on `0.0.0.0:PORT` and still refused, check two things: that the URL has the right port (`http://192.168.70.139:PORT`), and the firewall with `sudo ufw status`.

While you're on the server, run `nvidia-smi` too, so we can tell whether the driver install worked.

If you tell me how you normally start the API (the command or the service name) and which port it uses, I can give you the exact fix. I can also set it up as a systemd service so it starts automatically after every reboot.

You

is firewall blockig?

Claude

Probably not. A firewall usually *drops* traffic silently, and the browser then hangs and says "took too long to respond" (a timeout). "Refused to connect" usually means the request reached the machine but nothing was listening on that port. So the API most likely isn't running.

You can rule out the firewall in about 10 seconds:

```
sudo ufw status
```

- `Status: inactive`: the firewall isn't the problem.
- **Active**: check whether your port is allowed. If it isn't, allow it (replace 8000 with your port):

```
sudo ufw allow 8000/tcp
```

Then confirm whether the app is listening at all:

```
sudo ss -tlnp | grep 8000
```

If that prints nothing, the API isn't running, and starting it is the fix. Which port does your API use, and how do you usually start it?

You

```
(mvenv) anand@anand:/opt/ocr_api$ sudo ufw status
[sudo] password for anand:
Status: inactive
```

Claude

The firewall is off, so it isn't blocking anything. The problem is almost certainly that the API isn't running.

Run these two commands and paste the output:

```
sudo ss -tlnp
ls /opt/ocr_api
```

The first one shows which ports are listening. The second shows the files in your project, so I can see what the entry point is (for example `app.py`, `main.py`, or a run script).

If you already know how you start it, just start it again from the venv you're in:

```
cd /opt/ocr_api
python app.py          # or whatever your usual command is
```

Make sure it listens on `0.0.0.0`, not `127.0.0.1`, so it's reachable at 192.168.70.139. If it errors out on startup, paste the error here.

You

```
(mvenv) anand@anand:/opt/ocr_api$ sudo ss -tlnp
State     Recv-Q    Send-Q       Local Address:Port        Peer Address:Port    Process
LISTEN    0         511                0.0.0.0:80               0.0.0.0:*        users:(("nginx",pid=1390,fd=5),
("nginx",pid=1389,fd=5), ... ,("nginx",pid=1362,fd=5))
LISTEN    0         4096               0.0.0.0:22               0.0.0.0:*        users:(("sshd",pid=1361,fd=3),
("systemd",pid=1,fd=82))
LISTEN    0         200              127.0.0.1:5432             0.0.0.0:*        users:(("postgres",pid=1147,fd=6))
LISTEN    0         511              127.0.0.1:6379             0.0.0.0:*        users:(("redis-server",pid=1021,fd=6))
LISTEN    0         4096         127.0.0.53%lo:53               0.0.0.0:*        users:(("systemd-
resolve",pid=962,fd=15))
LISTEN    0         4096            127.0.0.54:53               0.0.0.0:*        users:(("systemd-
resolve",pid=962,fd=17))
LISTEN    0         511                 [::]:80                  [::]:*        users:(("nginx",pid=1390,fd=6),
("nginx",pid=1389,fd=6), ... ,("nginx",pid=1362,fd=6))
LISTEN    0         4096                [::]:22                  [::]:*        users:(("sshd",pid=1361,fd=4),
("systemd",pid=1,fd=83))
LISTEN    0         511                [::1]:6379                [::]:*        users:(("redis-server",pid=1021,fd=7))
(mvenv) anand@anand:/opt/ocr_api$ ls /opt/ocr_api
app                         ocr.db                      requirements.txt
Dockerfile                  ocr.db.bak-20261017-122752  storage
download_easyocr_models.py  ocr.db.bak-20261017-124559  wheels-win
logs                        reqr_2_gpu.txt
mvenv                       requirements
```

(The repeated nginx worker entries are shortened with "..." in this transcript.)

Claude

This tells us a lot. nginx is running on port 80, and PostgreSQL and Redis are up, but **no Python process (uvicorn, gunicorn, flask) is listening anywhere**. Your OCR API backend isn't running.

nginx is probably a reverse proxy in front of the API. If you'd opened `http://192.168.70.139` on port 80, you'd most likely have seen a "502 Bad Gateway" instead. "Refused" suggests the URL you used includes a port number, like `:8000`, and nothing is on that port right now.

Let's find out how the app is meant to be started. Run these and paste the output:

```
# Which port nginx forwards to
grep -R "proxy_pass" /etc/nginx/sites-enabled/

# Whether there's a service set up for the app
systemctl list-units --type=service --all | grep -iE "ocr|uvicorn|gunicorn|celery"

# How the Dockerfile starts it (gives us the exact command)
grep -E "CMD|ENTRYPOINT|EXPOSE" Dockerfile

# What's inside the app folder
ls app
```

If a service shows up in the second command (for example `ocr_api.service`), it probably failed at boot. In that case, also run `sudo systemctl status <name>` and `sudo journalctl -u <name> -n 50`. The error in that log will likely be the real cause, possibly related to the new NVIDIA driver or Paddle.

You

in my server, i ran. i did ssh, and connected to server. how can i check from my system?

```
uvicorn app.main:app --reload --port 8000
```

Claude

That's the cause. Without `--host`, uvicorn listens only on `127.0.0.1` (the server itself), so your laptop's connection gets refused. Tell it to listen on all interfaces.

Stop it with Ctrl+C, then run:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Now open this in the browser on your own computer:

```
http://192.168.70.139:8000/docs
```

If it's a FastAPI app, `/docs` shows the interactive API page. Your laptop needs to be on the same network (or VPN) as the server, which it seems to be, since the connection reached the server.

Option 2: keep it private and use an SSH tunnel. Leave uvicorn as it was and run this on your own computer:

```
ssh -L 8000:localhost:8000 anand@192.168.70.139
```

While that SSH session is open, `http://localhost:8000/docs` on your computer reaches the API.

Two things to know:

- **The app stops when you close the SSH session.** To keep it running after you log out, run it inside `tmux`: type `tmux`, start uvicorn, then press Ctrl+B, then D to detach. For something permanent that also survives reboots, I can write you a systemd service.
- **`--reload` is for development.** It watches files and restarts the server, so drop it once things are working.

Your nginx on port 80 may already be configured to forward to port 8000. If so, `http://192.168.70.139` without a port would also work once uvicorn is running. Running `grep -R proxy_pass /etc/nginx/sites-enabled/` will tell you.

You

i installed nginx. i need to get this service as http://192.168.70.139.ocr_api, give me the steos

Claude

A URL can't add a name to an IP like `192.168.70.139.ocr_api`; the browser would treat that whole thing as a hostname. The usual way is a path:

```
http://192.168.70.139/ocr_api
```

Here's how to set that up. In these steps, nginx listens on port 80 and forwards `/ocr_api` to uvicorn, and a systemd service keeps uvicorn running after logout and reboot.

1. Stop the uvicorn you started manually

Press Ctrl+C in that terminal. Otherwise port 8000 will be busy.

2. Create a systemd service for the API

```
sudo nano /etc/systemd/system/ocr_api.service
```

Paste:

```
[Unit]
Description=OCR API (uvicorn)
After=network.target postgresql.service redis-server.service

[Service]
User=anand
WorkingDirectory=/opt/ocr_api
ExecStart=/opt/ocr_api/mvenv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --root-path /ocr_api
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

`--host 127.0.0.1` is deliberate: only nginx talks to uvicorn, and nothing outside the server can reach port 8000 directly. `--root-path /ocr_api` lets the `/docs` page work under the subpath.

Start it, and enable it so it starts at every boot:

```
sudo systemctl daemon-reload
sudo systemctl enable --now ocr_api
sudo systemctl status ocr_api
```

If the status shows a failure, run `sudo journalctl -u ocr_api -n 50` and paste the output here.

3. Configure nginx

Check what's already enabled:

```
ls /etc/nginx/sites-enabled/
```

If the only entry is `default`, replace it with your own site:

```
sudo rm /etc/nginx/sites-enabled/default
sudo nano /etc/nginx/sites-available/ocr_api
```

Paste:

```
server {
    listen 80 default_server;
    server_name 192.168.70.139;

    client_max_body_size 50M;        # allow large image/PDF uploads

    location /ocr_api/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;     # OCR can be slow
    }

    location = /ocr_api {
        return 301 /ocr_api/;
    }
}
```

The trailing `/` in `proxy_pass http://127.0.0.1:8000/;` matters. It strips `/ocr_api` from the path before forwarding, so your app keeps its normal routes.

Enable the site, test the config, and reload nginx:

```
sudo ln -s /etc/nginx/sites-available/ocr_api /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

4. Test from your computer

```
http://192.168.70.139/ocr_api/docs
```

Your endpoints are now at `http://192.168.70.139/ocr_api/<route>`. For example, `/upload` becomes `/ocr_api/upload`.

If you'd rather have a name without the IP, like `http://ocr-api.local`, that needs either a DNS entry on your network or a line in the hosts file of each client machine. I can give you those steps instead. Also, if `ls /etc/nginx/sites-enabled/` shows other sites besides `default`, paste the list first so we don't break them.

You

i ve build nextjs application. now i need to deploy in ubuntu, and i need to access the application as http://192.168.70.139/ocr

below is the next config file. make the changes

```
import type { NextConfig } from "next";

// Set when IIS hosts the app under a sub-path (IIS application), e.g. BASE_PATH=/ocr.
// Leave unset when the app is the root of its own IIS site.
const basePath = process.env.BASE_PATH?.replace(/\/+$/, "") || undefined;

const nextConfig: NextConfig = {
  // Static export: `npm run build` writes plain HTML/JS/CSS to ./out for IIS.
  output: "export",
  basePath,
  // Emit /history/index.html etc. so IIS serves routes via its default document.
  trailingSlash: true,
  images: { unoptimized: true },
  reactStrictMode: true,
};

export default nextConfig;
```

Claude

Since the app is a static export, you don't need Node running on the server: nginx serves the built files directly, like IIS did. Here's the updated config, with `/ocr` as the default base path so you don't need to set an environment variable at build time:

```
import type { NextConfig } from "next";

// The app is served by nginx under a sub-path: http://<server>/ocr
// Override with BASE_PATH (e.g. BASE_PATH=/other); set BASE_PATH="" to serve from the root.
const basePath =
  (process.env.BASE_PATH ?? "/ocr").replace(/\/+$/, "") || undefined;

const nextConfig: NextConfig = {
  // Static export: `npm run build` writes plain HTML/JS/CSS to ./out for nginx.
  output: "export",
  basePath,
  // Emit /history/index.html etc. so nginx serves routes via index.html.
  trailingSlash: true,
  images: { unoptimized: true },
  reactStrictMode: true,
};

export default nextConfig;
```

1. Point the frontend at the API

Your API is now at `http://192.168.70.139/ocr_api`. If the frontend reads the API URL from an environment variable (for example `NEXT_PUBLIC_API_URL`), set it before building:

```
# .env.production
NEXT_PUBLIC_API_URL=/ocr_api
```

A relative `/ocr_api` works because the frontend and API share the same host, and it also avoids CORS issues. If the URL is

hardcoded in your code (such as `http://localhost:8000`), change it there instead. `NEXT_PUBLIC_*` values are baked in at build time, so set them before you run the build.

2. Build

You can build on your own machine, or on the server if Node is installed there:

```
npm ci
npm run build        # produces ./out
```

If you build on Windows, copy the `out` folder to the server, for example with `scp -r out anand@192.168.70.139:/tmp/ocr_out`.

3. Put the files where nginx can serve them

```
sudo mkdir -p /var/www/ocr_web/ocr
sudo rsync -a --delete out/ /var/www/ocr_web/ocr/      # or /tmp/ocr_out/ if you copied it over
```

The extra `ocr` folder level is intentional. Inside it, the folder structure matches the URL path, which keeps the nginx config simple.

4. Add the frontend to your nginx site

```
sudo nano /etc/nginx/sites-available/ocr_api
```

Add these blocks inside the existing `server { ... }`, next to the `/ocr_api/` block:

```
    # Next.js static frontend
    location /ocr/ {
        root /var/www/ocr_web;
        index index.html;
        try_files $uri $uri/ $uri.html /ocr/404.html;
    }

    location /ocr/_next/static/ {
        root /var/www/ocr_web;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location = /ocr {
        return 301 /ocr/;
    }
```

Test the config and reload nginx:

```
sudo nginx -t
sudo systemctl reload nginx
```

5. Open it

```
http://192.168.70.139/ocr/
```

When you update the app later: rebuild, run the `rsync` from step 3 again, and you're done. No nginx reload is needed.

If a page loads but styles or scripts return 404, the base path wasn't applied at build time. Open DevTools → Network: the asset URLs should start with `/ocr/_next/...`. If API calls fail, check that they go to `/ocr_api/...`.

You

explain this step

4. Add the frontend to your nginx site

Claude

Step 4 tells nginx two things: when a browser asks for anything under `/ocr/`, look for the file in `/var/www/ocr_web`; and if the

exact file isn't there, try a few sensible alternatives. Here's what each block does.

Where it goes

You already have one `server { ... }` block for port 80, the one that forwards `/ocr_api/` to uvicorn. One server block can handle many URL paths, so the frontend blocks go inside it, next to the API block:

```
server {
    listen 80 default_server;
    ...

    location /ocr_api/ { ... }     # → backend (uvicorn)
    location /ocr/ { ... }         # → frontend files   (new)
    location /ocr/_next/static/ { ... }              # (new)
    location = /ocr { ... }                          # (new)
}
```

nginx picks the `location` that best matches the request path. `/ocr_api/...` and `/ocr/...` are different prefixes, so they don't interfere with each other.

Block 1: serving the pages

```
location /ocr/ {
    root /var/www/ocr_web;
    index index.html;
    try_files $uri $uri/ $uri.html /ocr/404.html;
}
```

- **`location /ocr/`** matches every URL that starts with `/ocr/`.
- **`root /var/www/ocr_web`** tells nginx to find files by appending the full URL path to this folder. A request for `/ocr/history/` maps to `/var/www/ocr_web/ocr/history/`. That's why step 3 copied the build into `/var/www/ocr_web/ocr`: the folder layout mirrors the URL.
- **`index index.html`**: when the request is for a folder, nginx serves the `index.html` inside it.
- **`try_files`** is a fallback list that nginx tries in order:
  1. `$uri`: the exact file, such as `/ocr/logo.png`.
  2. `$uri/`: a folder with that name, served through its `index.html`. Thanks to `trailingSlash: true`, Next.js builds every page this way, so `/ocr/history/` → `ocr/history/index.html`.
  3. `$uri.html`: covers a link that omits the trailing slash, such as `/ocr/history` → `history.html`, if it exists.
  4. `/ocr/404.html`: if nothing matches, nginx shows Next.js's own 404 page instead of nginx's plain error page.

Block 2: caching the JS and CSS

```
location /ocr/_next/static/ {
    root /var/www/ocr_web;
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

Next.js puts its JavaScript and CSS in `_next/static/`, with a content hash in every filename (for example `main-a8f3c2.js`). When the code changes, the filename changes too, so it's safe to let browsers cache these files for a year. Pages then load faster after the first visit. This block is optional; the app works without it.

Block 3: the redirect

```
location = /ocr {
    return 301 /ocr/;
}
```

The `=` means "exactly `/ocr`, nothing more." When someone types `http://192.168.70.139/ocr` without the trailing slash, this sends them to `/ocr/`, where block 1 takes over. Without this block, that URL would return a 404.

A request, start to finish

When you open `http://192.168.70.139/ocr/history/`:

1. nginx receives `/ocr/history/` on port 80 and matches `location /ocr/`.
2. `$uri` (`/var/www/ocr_web/ocr/history/`) is a folder, not a file, so nginx moves on.

3. `$uri/` applies the index rule and finds `/var/www/ocr_web/ocr/history/index.html`, which it serves.
4. That page loads `/ocr/_next/static/...js`. Those requests match block 2, are served from disk, and are cached.
5. When the page calls `/ocr_api/...`, the request matches the API block and is forwarded to uvicorn.

After editing

```
sudo nginx -t                 # checks the syntax; don't skip it
sudo systemctl reload nginx   # applies the change without dropping connections
```

If `nginx -t` reports an error, paste it here. It's usually a missing `;` or `}`.

You

is this correct?

```
server {
    listen 80 default_server;
    server_name 192.168.70.139;

    client_max_body_size 50M;        # allow large image/PDF uploads

    location /ocr_api/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;     # OCR can be slow
    }

    location = /ocr_api {
        return 301 /ocr_api/;
    }

    # Next.js static frontend
    location /ocr/ {
        root /var/www/ocr_web;
        index index.html;
        try_files $uri $uri/ $uri.html /ocr/404.html;
    }

    location /ocr/_next/static/ {
        root /var/www/ocr_web;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location = /ocr {
        return 301 /ocr/;
    }
}
```

Claude

Yes, it's correct. The paths, the `root` folder, the `try_files` order, and the location blocks all fit together. The mixed tabs and spaces before `# Next.js static frontend` don't matter to nginx.

One optional improvement: as written, a missing page shows your 404 page but sends HTTP status **200** instead of 404. That's because the last `try_files` entry is treated as a normal page. To get a proper 404 status, change the `/ocr/` block to this:

```
    location /ocr/ {
        root /var/www/ocr_web;
        index index.html;
        try_files $uri $uri/ $uri.html =404;
        error_page 404 /ocr/404.html;
    }
```

Visitors see the same page either way. The difference only matters for monitoring tools and crawlers.

To apply and check it:

```
sudo nginx -t
sudo systemctl reload nginx
```

Then test from the server:

```
curl -I http://127.0.0.1/ocr/          # expect: 200 OK
curl -I http://127.0.0.1/ocr           # expect: 301 → /ocr/
curl -I http://127.0.0.1/ocr_api/docs  # expect: 200 OK (if the API service is running)
```

If `/ocr/` returns 404 or 403, check that the build landed in the right place with `ls /var/www/ocr_web/ocr/`. You should see `index.html` and a `_next` folder directly inside it, not inside an extra `out` folder.

You

```
anand@anand:~$ curl -I http://127.0.0.1/ocr/
HTTP/1.1 500 Internal Server Error
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 28 Oct 2026 17:52:00 GMT
Content-Type: text/html
Content-Length: 186
Connection: close

anand@anand:~$ curl -I http://127.0.0.1/ocr
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 28 Oct 2026 17:52:25 GMT
Content-Type: text/html
Content-Length: 178
Location: http://127.0.0.1/ocr/
Connection: keep-alive
```

Claude

The redirect works. The 500 on `/ocr/` almost always means nginx couldn't find or read `index.html`, then went to the fallback `/ocr/404.html`, couldn't find that either, and ended up in a loop. nginx stops the loop with a 500 error.

Two commands will show which it is:

```
sudo tail -n 20 /var/log/nginx/error.log
ls -la /var/www/ocr_web/ocr/
```

Here's what the likely results mean.

`rewrite or internal redirection cycle` in the log, and the folder is empty or missing: the build files aren't there yet. Copy them in, adjusting the source path to wherever your `out` folder is:

```
sudo mkdir -p /var/www/ocr_web/ocr
sudo rsync -a --delete /path/to/out/ /var/www/ocr_web/ocr/
```

The trailing `/` on `out/` matters: it copies the contents of `out`, not the folder itself.

The folder contains `out/` instead of `index.html`: the files are one level too deep. Fix it with:

```
sudo mv /var/www/ocr_web/ocr/out/* /var/www/ocr_web/ocr/
sudo rmdir /var/www/ocr_web/ocr/out
```

`Permission denied` in the log: nginx (which runs as the `www-data` user) can't read the files. Fix the permissions:

```
sudo chown -R www-data:www-data /var/www/ocr_web
sudo find /var/www/ocr_web -type d -exec chmod 755 {} \;
sudo find /var/www/ocr_web -type f -exec chmod 644 {} \;
```

Once `index.html` and `_next/` sit directly in `/var/www/ocr_web/ocr/`, run `curl -I http://127.0.0.1/ocr/` again and you should get `200 OK`. If not, paste the error log lines here.

You

```
anand@anand:~$ sudo tail -n 20 /var/log/nginx/error.log
2026/10/28 12:30:05 [notice] 6595#6595: using inherited sockets from "5;6;"
2026/10/28 16:37:06 [alert] 2496#2496: *3 open socket #7 left in connection 2
2026/10/28 16:37:06 [alert] 2496#2496: aborting
2026/10/28 17:50:44 [error] 2247#2247: *49 rewrite or internal redirection cycle while internally redirecting to
"/ocr/404.html", client: 192.168.70.137, server: 192.168.70.139, request: "GET /ocr/ HTTP/1.1", host: "192.168.70.139"
2026/10/28 17:50:45 [error] 2247#2247: *50 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or
directory), client: 192.168.70.137, server: 192.168.70.139, request: "GET /favicon.ico HTTP/1.1", host:
"192.168.70.139", referrer: "http://192.168.70.139/ocr/"
2026/10/28 17:50:58 [error] 2247#2247: *50 rewrite or internal redirection cycle while internally redirecting to
"/ocr/404.html", client: 192.168.70.137, server: 192.168.70.139, request: "GET /ocr/ HTTP/1.1", host: "192.168.70.139"
2026/10/28 17:52:00 [error] 2249#2249: *52 rewrite or internal redirection cycle while internally redirecting to
"/ocr/404.html", client: 127.0.0.1, server: 192.168.70.139, request: "HEAD /ocr/ HTTP/1.1", host: "127.0.0.1"
```

Claude

The log confirms it: "rewrite or internal redirection cycle ... to /ocr/404.html". nginx can't find `index.html` or `404.html` in `/var/www/ocr_web/ocr/`, so the build files are missing or in the wrong place. The nginx config itself is fine.

Check what's there:

```
ls -la /var/www/ocr_web/ocr/
```

Then fix it based on what you see.

Empty, or "No such file or directory": copy the build in. Where is your `out` folder? If you built on the server, it's inside your Next.js project folder:

```
sudo mkdir -p /var/www/ocr_web/ocr
sudo rsync -a --delete /path/to/your-nextjs-project/out/ /var/www/ocr_web/ocr/
```

If you built on your Windows machine, copy it over from there first. In PowerShell:

```
scp -r .\out anand@192.168.70.139:/tmp/ocr_out
```

Then, on the server:

```
sudo rsync -a --delete /tmp/ocr_out/ /var/www/ocr_web/ocr/
```

You see an `out` folder (not `index.html`): the files are one level too deep:

```
sudo rsync -a /var/www/ocr_web/ocr/out/ /var/www/ocr_web/ocr/
sudo rm -rf /var/www/ocr_web/ocr/out
```

Either way, finish with:

```
sudo chmod -R a+rX /var/www/ocr_web
ls /var/www/ocr_web/ocr/          # should show index.html, 404.html, _next/
curl -I http://127.0.0.1/ocr/     # should be 200 OK
```

The `favicon.ico` error in the log is harmless. Browsers always ask for `/favicon.ico` at the site root, but your app's icon lives under `/ocr/`. If you want to silence it, add this to the server block:

```
    location = /favicon.ico {
        alias /var/www/ocr_web/ocr/favicon.ico;
        log_not_found off;
    }
```

You

my path is /var/www/ocr

```
server {
    listen 80 default_server;
    server_name 192.168.70.139;

    client_max_body_size 50M;        # allow large image/PDF uploads

    location /ocr_api/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;     # OCR can be slow
    }

    location = /ocr_api {
        return 301 /ocr_api/;
    }

    # Next.js static frontend
    location /ocr/ {
        root /var/www;
        index index.html;
        try_files $uri $uri/ $uri.html /ocr/404.html;
    }

    location /ocr/_next/static/ {
        root /var/www;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location = /ocr {
        return 301 /ocr/;
    }
}
```

Claude

Yes, that's correct for files in `/var/www/ocr`. With `root /var/www`, nginx appends the URL path, so `/ocr/` maps to `/var/www/ocr/` and `/ocr/history/` maps to `/var/www/ocr/history/index.html`.

The build files have to sit directly inside that folder. Check:

```
ls /var/www/ocr/
```

You should see `index.html`, `404.html` and `_next/`. If you see `out/` instead, move its contents up one level:

```
sudo rsync -a /var/www/ocr/out/ /var/www/ocr/
sudo rm -rf /var/www/ocr/out
```

Then make the files readable by nginx, apply the config, and test:

```
sudo chmod -R a+rX /var/www/ocr
sudo nginx -t
sudo systemctl reload nginx
curl -I http://127.0.0.1/ocr/
```

You should get `200 OK`. If you also add the favicon block from earlier, point it at the new path: `alias /var/www/ocr/favicon.ico;`.